

			glob-parent
1 8	2 7	3 9	4 5
6		Extract the non-magic parent path from a glob string. Usage <pre>var globParent = require('glob-parent'); globParent('path/to/*.js'); // 'path/to' globParent('/root/path/to/*.js'); // '/root/path/to' globParent('/*.js'); // ''</pre>	<pre>globParent('*.js'); // '.' globParent('**/*.js'); // '.' globParent('path/{to,from}'); // 'path' globParent('path/!(to from)'); // 'path' globParent('path/(to from)'); // 'path' globParent('path/+(to from)'); // 'path' globParent('path/*(to from)'); // 'path'</pre>
16 escaped if you want them to be treated as regular path characters: ? regular path characters: ? (question mark) unless used as a path segment alone * (asterisk) (pipe) (opening parenthesis)) (closing parenthesis) { (opening curly brace) } (closing curly brace) [(opening bracket)] (closing bracket)	15 options <pre>{ // Disables the automatic conversion of slashes for Windows flipBackslashes: true }</pre> Escaping The following characters have special significance in glob patterns and must be escaped	14 API Takes a string and returns the part of the path before the glob begins. Be aware of Escaping rules and Limitations below. globParent(maybeGlobString, [options])	13 <pre>globParent('path/@(to from)'); // 'path' globParent('path/**/*'); // 'path' // if provided a non-glob path, returns the nearest dir globParent('path/foo/bar.js'); // 'path/foo' globParent('path/foo/'): // 'path/foo' globParent('path/'): // 'path' (see issue #3 for details)</pre>

Example globParent('foo/[bar]/') // 'foo' globParent('foo/\\[bar]/') // 'foo/[bar]' Limitations Braces & Brackets This library attempts a quick and imperfect method of determining which path parts have glob magic without fully parsing/lexing the 17	pattern. There are some advanced use cases that can trip it up, such as nested braces where the outer pair is escaped and the inner one contains a path separator. If you find yourself in the unlikely circumstance of being affected by this or need to ensure higher-fidelity glob handling in your library, it is recommended that you pre-process your input with expand-braces and/or expand-brackets. 18	Windows Backslashes are not valid path separators for globs. If a path with backslashes is provided anyway, for simple cases, glob-parent will replace the path separator for you and return the non-glob parent path (now with forward-slashes, which are still valid as Windows path separators). This cannot be used in conjunction with escape characters. 19	// BAD globParent('C:\\Program Files \\(x86\\)*.ext') // 'C:/Program Files /(x86/') // GOOD globParent('C:/Program Files\\(x86\\)/*.ext') // 'C:/Program Files (x86)' If you are using escape characters for a pattern without path parts (i.e. relative to 20
24	23	22 ISC License	21 // BAD parent. cwd), prefix with ./ to avoid confusing glob- globParent('foo \\[bar]') // 'foo ' globParent('foo \\[bar]*') // 'foo ' // GOOD globParent('./foo \\[bar]') // 'foo [bar]' globParent('./foo \\[bar]*') // 'foo '[bar]'
25	26	27	28
32	31	30	29